

SOKOBAN

SOFIA DEMCHUK & YULIYA LOS



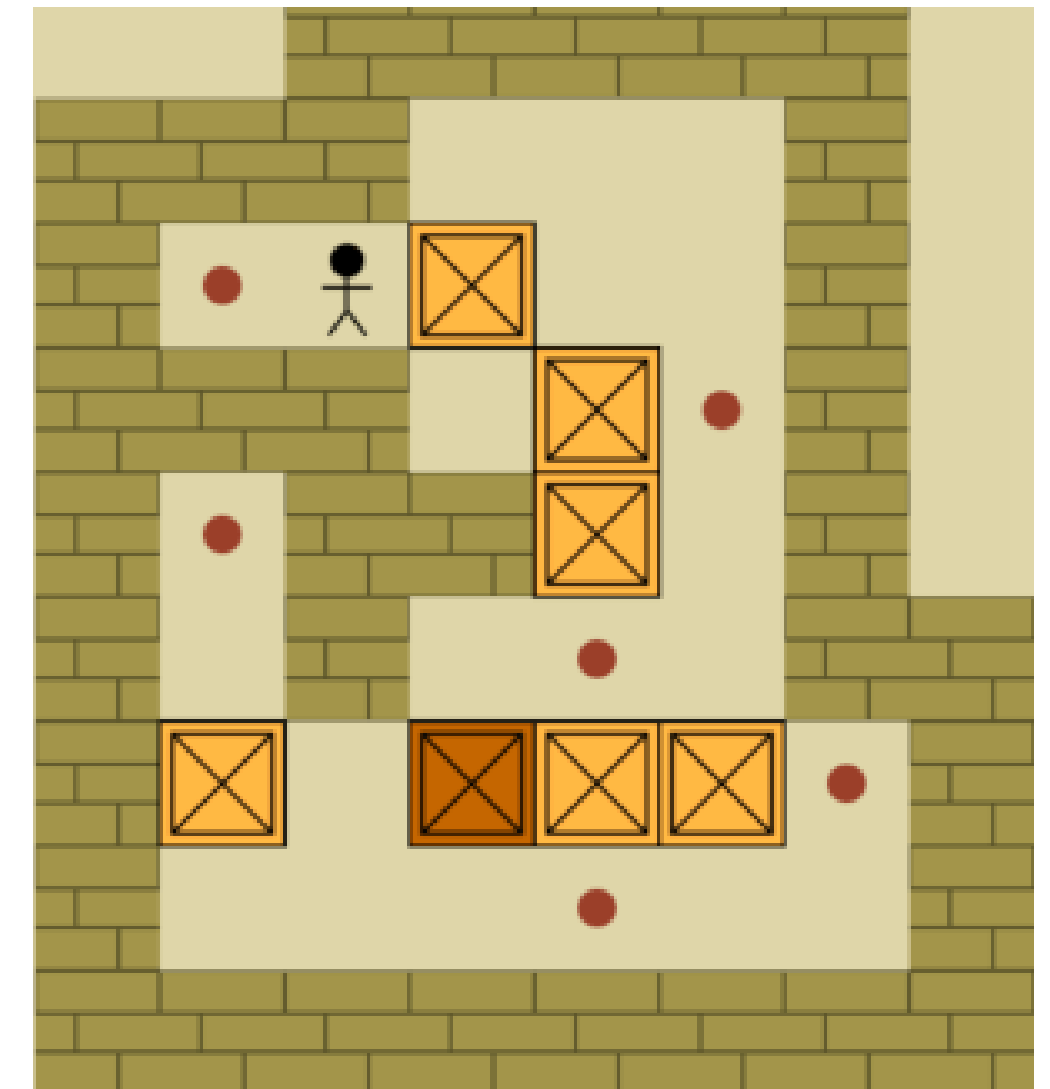
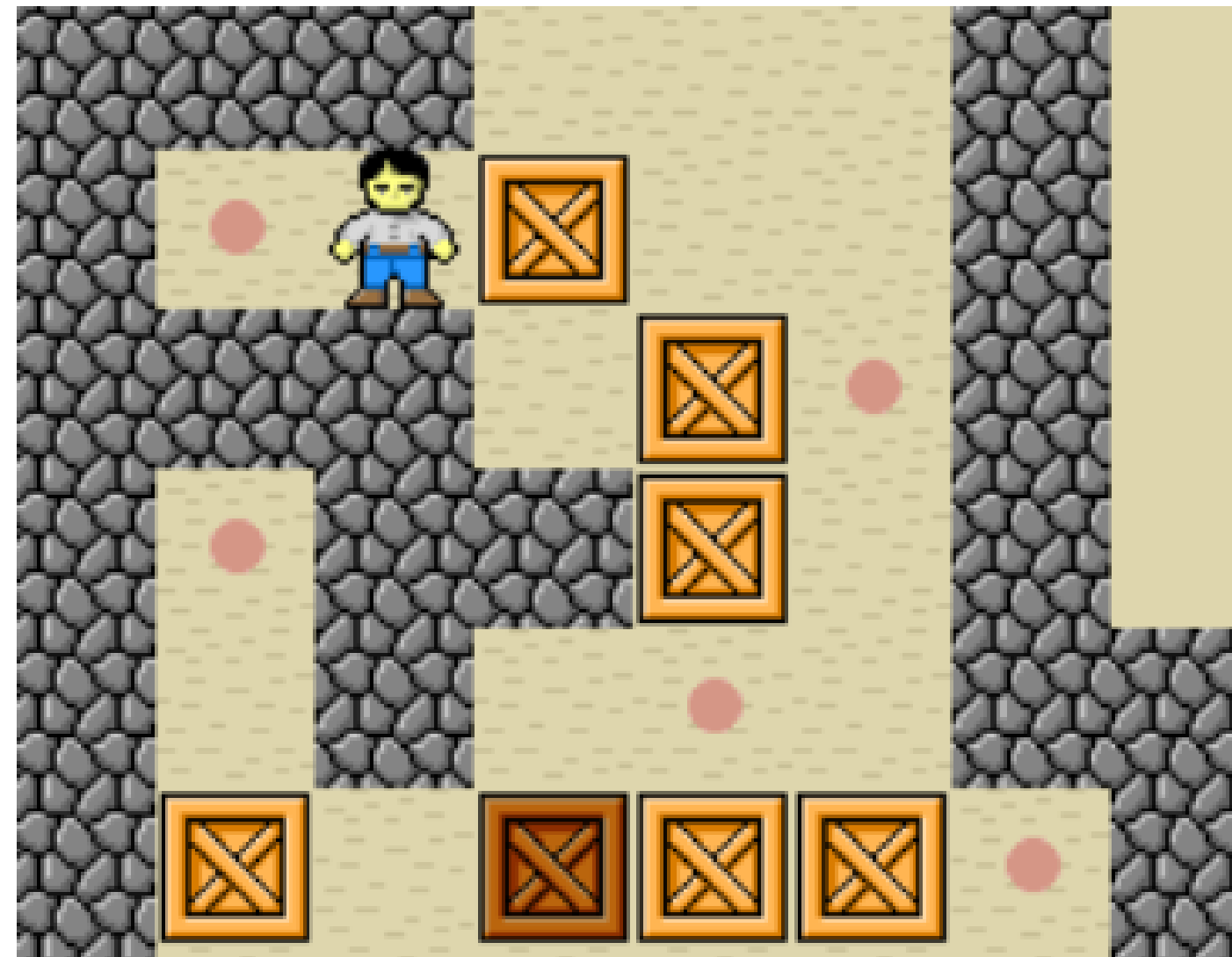
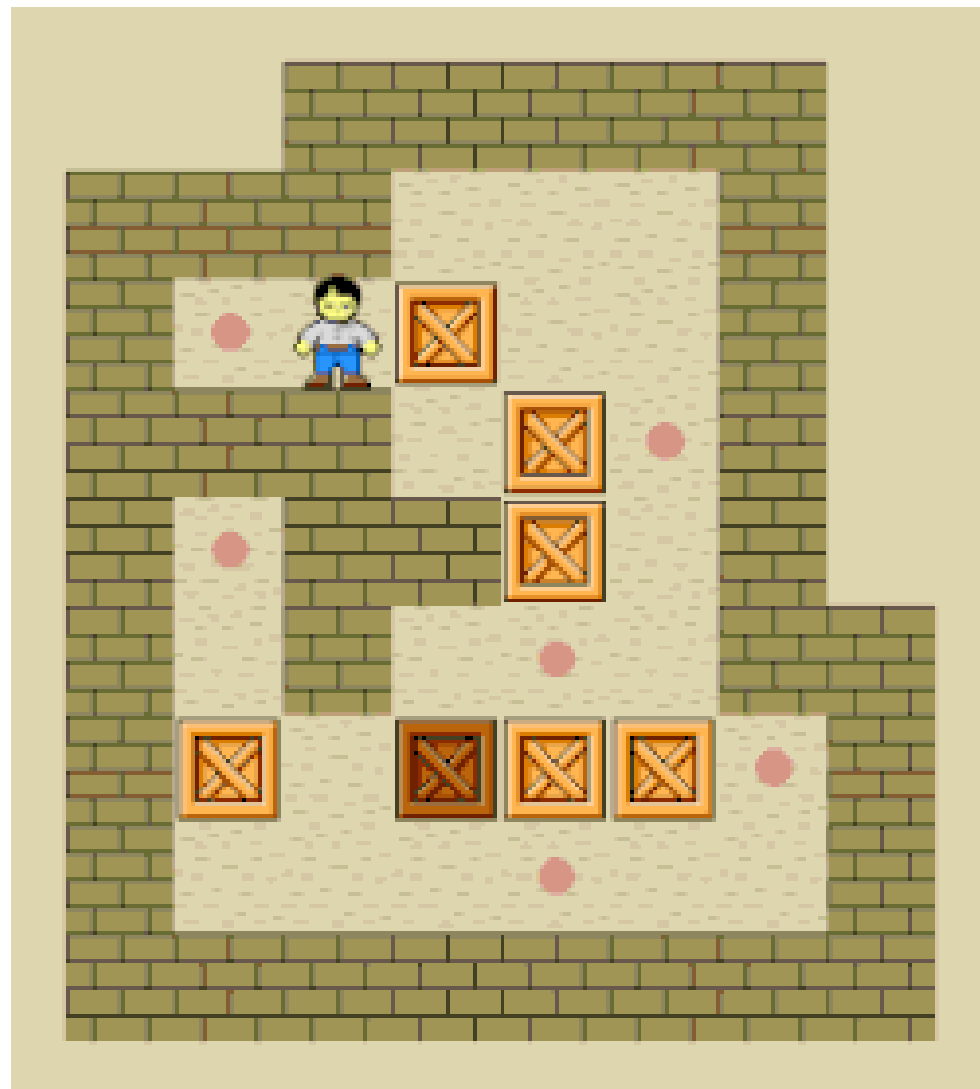
MIAGE M1 UNIVERSITE DE LILLE

<https://github.com/soniaa28/Myg/tree/dev>

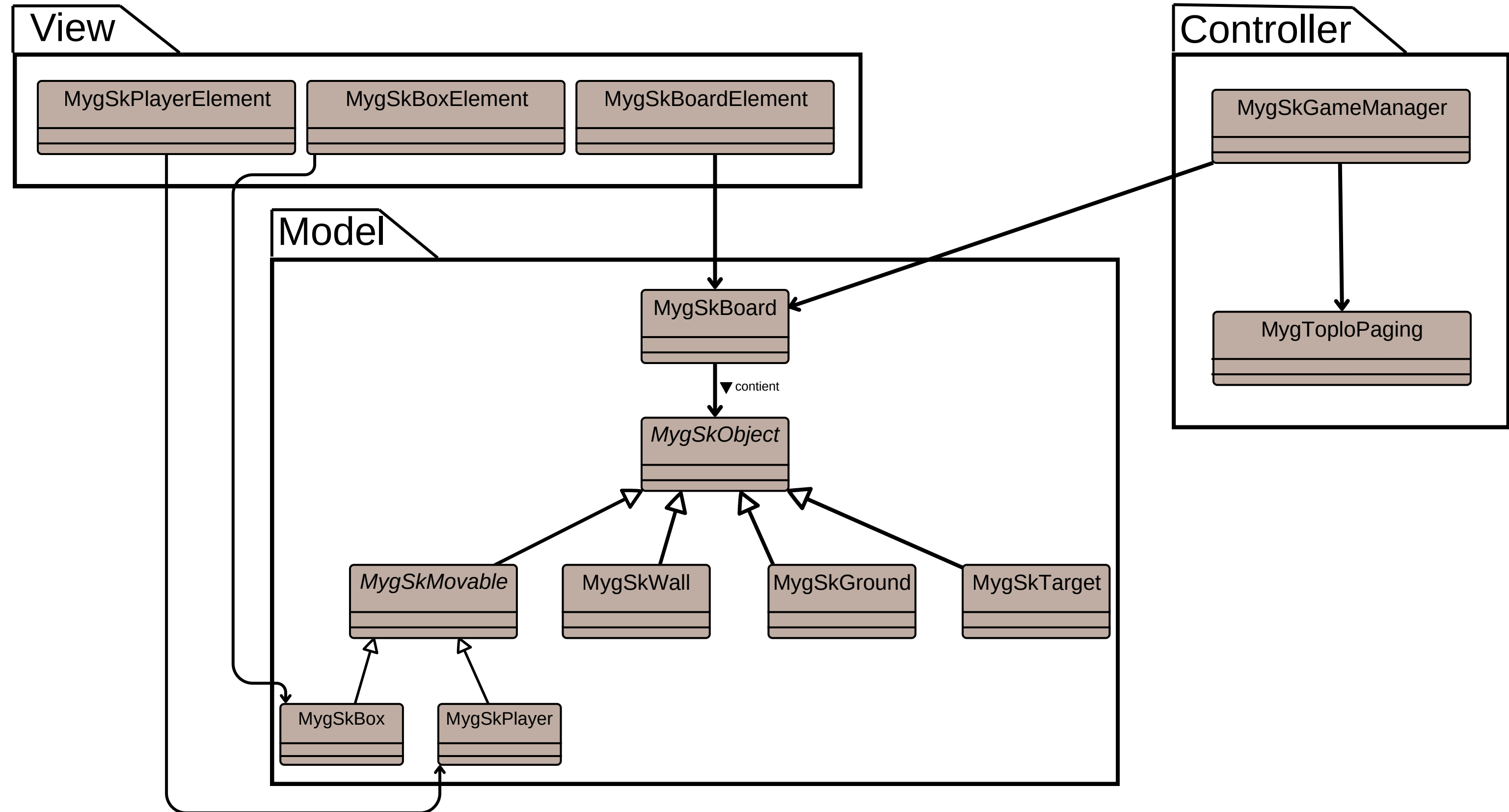
Qu'est-ce que c'est que le jeu Sokoban ?



Sokoban est un jeu de réflexion classique dans lequel le joueur doit pousser des caisses pour les placer sur des cibles. Le personnage peut pousser une caisse mais ne peut jamais la tirer, ce qui crée des situations de blocage et oblige à réfléchir plusieurs coups à l'avance.



Design initial du projet



Objectifs des nouvelles fonctionnalités

1.Teleport title. Given two teleport tiles, implement teleport so that when the player move it, teleports the player to another wrap tile.

2.Counting moves and push. Introduce the display of moves.

3.Numbered Target. Introduce number on the target and make sure that the box should be moved in order on the target.

4.Paired Target/Box. Introduce pairs of target/box where each box can only go on a target. A version can mix paired and unpaired boxes.



Compteur de mouvements

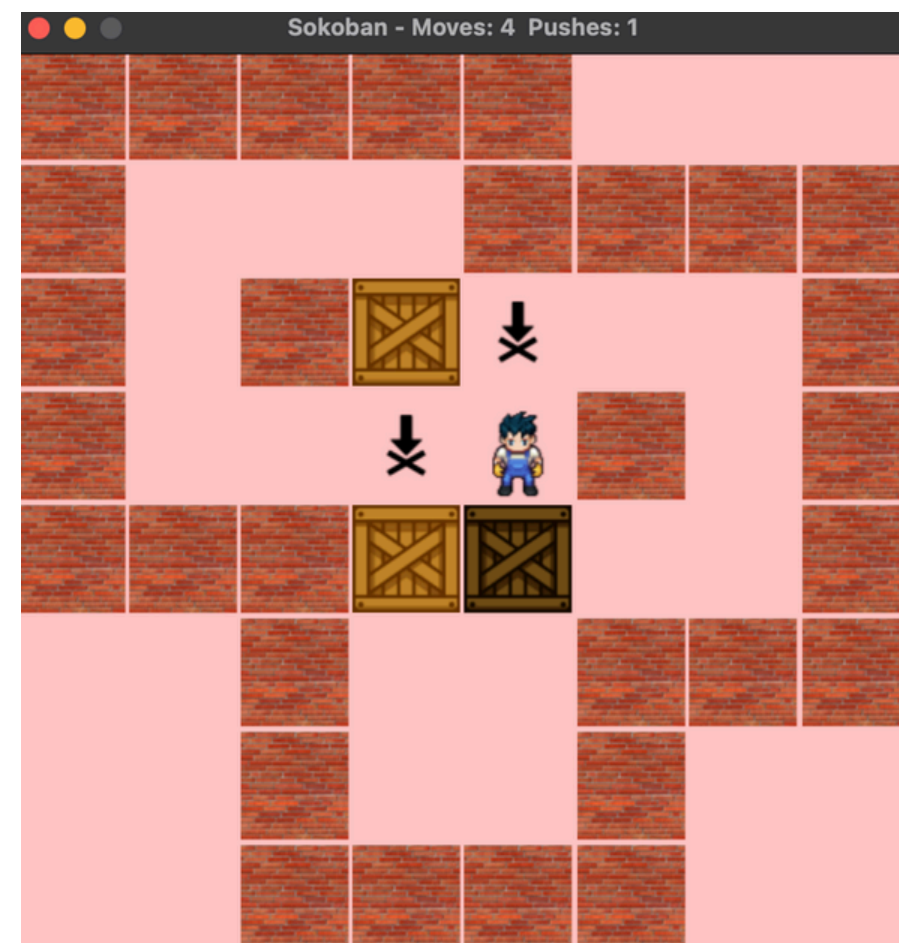
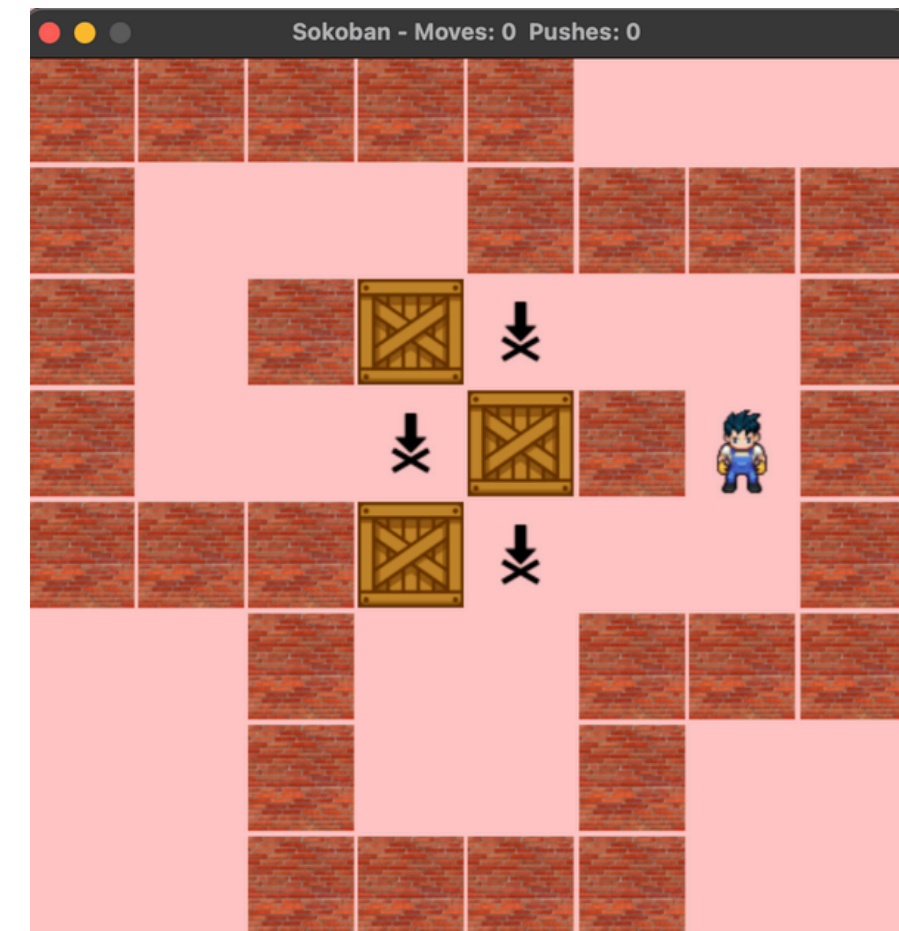
Conception

- Ajout d'un compteur intégré dans la logique du jeu (déplacements (moveCount)+ poussées (pushCount)).
- Mise à jour automatique lors de chaque action du joueur.
- Affichage dynamique dans l'interface du jeu.
- Réinitialisation du compteur à chaque début de niveau.

Patterns utilisés

Observer-like behaviour (réaction automatique aux actions du joueur). Le compteur se met à jour dès que le joueur effectue un déplacement.

Single Responsibility Principle. Le compteur est isolé dans une responsabilité claire : mesurer l'activité du joueur.



Teleport : Design après

Conception

- Nouveau caractère dans le DSL : \$T.
- Classe dédiée : MygSkTeleport.
- Gestion du saut lors du déplacement du joueur.
- Rendu visuel personnalisé (backgroundRepresentation).

Patterns utilisés

Factory Method / Extension du DSL

Le parseur reconnaît automatiquement \$T → crée MygSkTeleport.

Polymorphisme

Le téléporteur se comporte comme un objet du plateau, mais avec un effet spécial.

```
#####  
#      ###      #  
#  $      .      #  
#   #   @   #  
#      ###      #  
#####
```



```
#####  
#      ###      #  
#  $      .      #  
#   #   @   #  
#   T   ###   T   #  
#####
```

DSL de projet :

— Mur

@ — Joueur

(espace) — Sol

\$ — Boîte standard

. — Cible standard

1



T



T

3



T

T

2



T

4

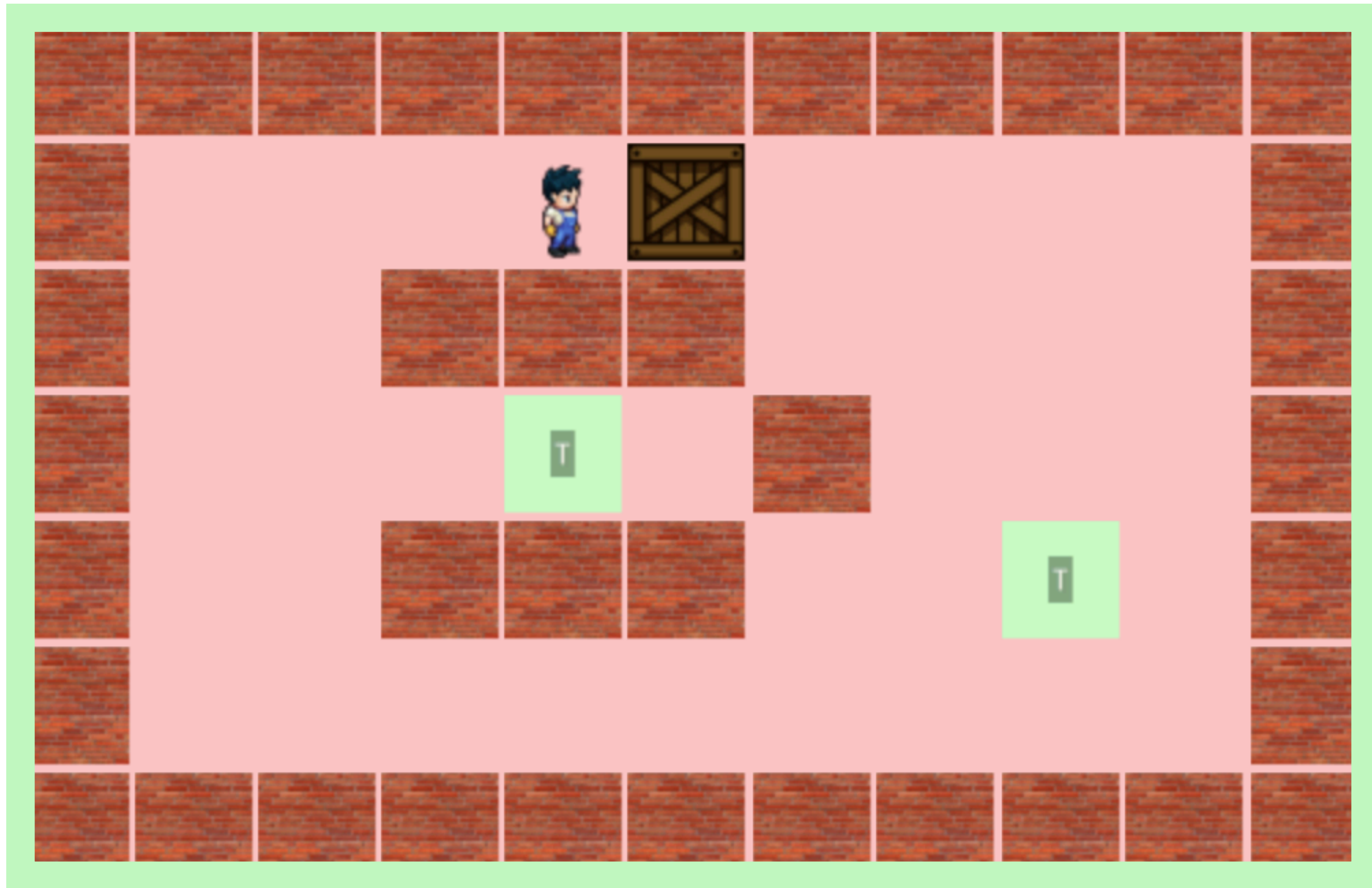


T

T



Victory!



Paired Target/Box



Conception

MygSkPairedBox et MygSkPairedTarget.
Nouvelle logique dans isOnCorrectTarget.
Changement d'apparence selon l'état :
violet (au sol)
brun foncé (bien placée)

Patterns utilisés

Polymorphisme + Tell-Don't-Ask

La boîte détermine elle-même si elle est sur la bonne cible.

State Pattern

État de la boîte : normale, mal placée, correctement placée

```
#####  
#      .      #  
#     $      #  
#    @       #  
#           #  
#####
```



```
#####  
#   1   .   #  
#  2$    #  
#   @    #  
#        #  
#####
```

1 - pairedTarget
2 - pairedBox

1



2



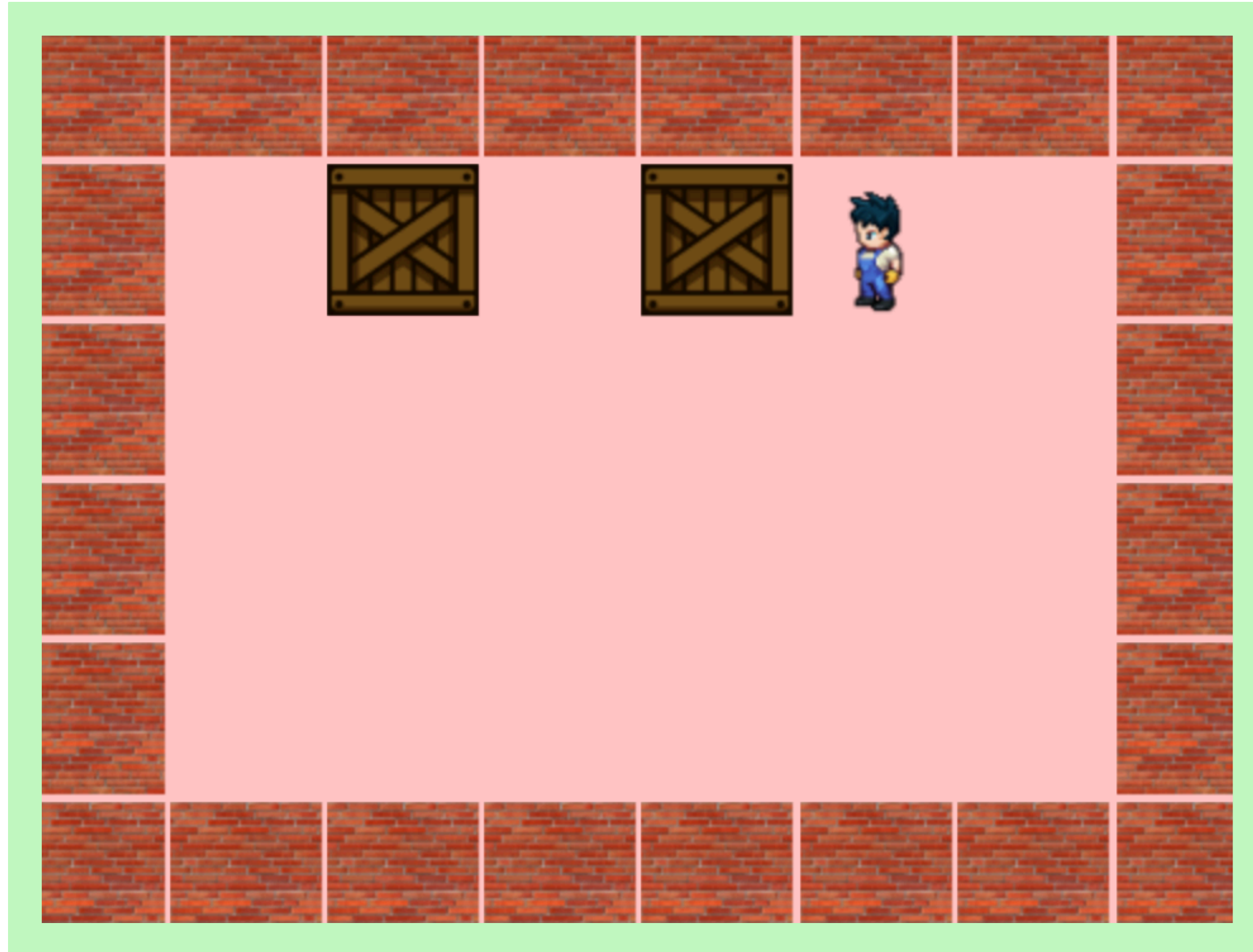
3



4



Victory!



Numbered Target

Conception

Nouvelles classes : MygSkBoxA, MygSkBoxB, MygSkBoxC (+ cibles).

Méthode updateNumberedBoxAfterMove: dans MygSkBoard.

Règle : A doit être placée avant B, puis C.

Patterns utilisés

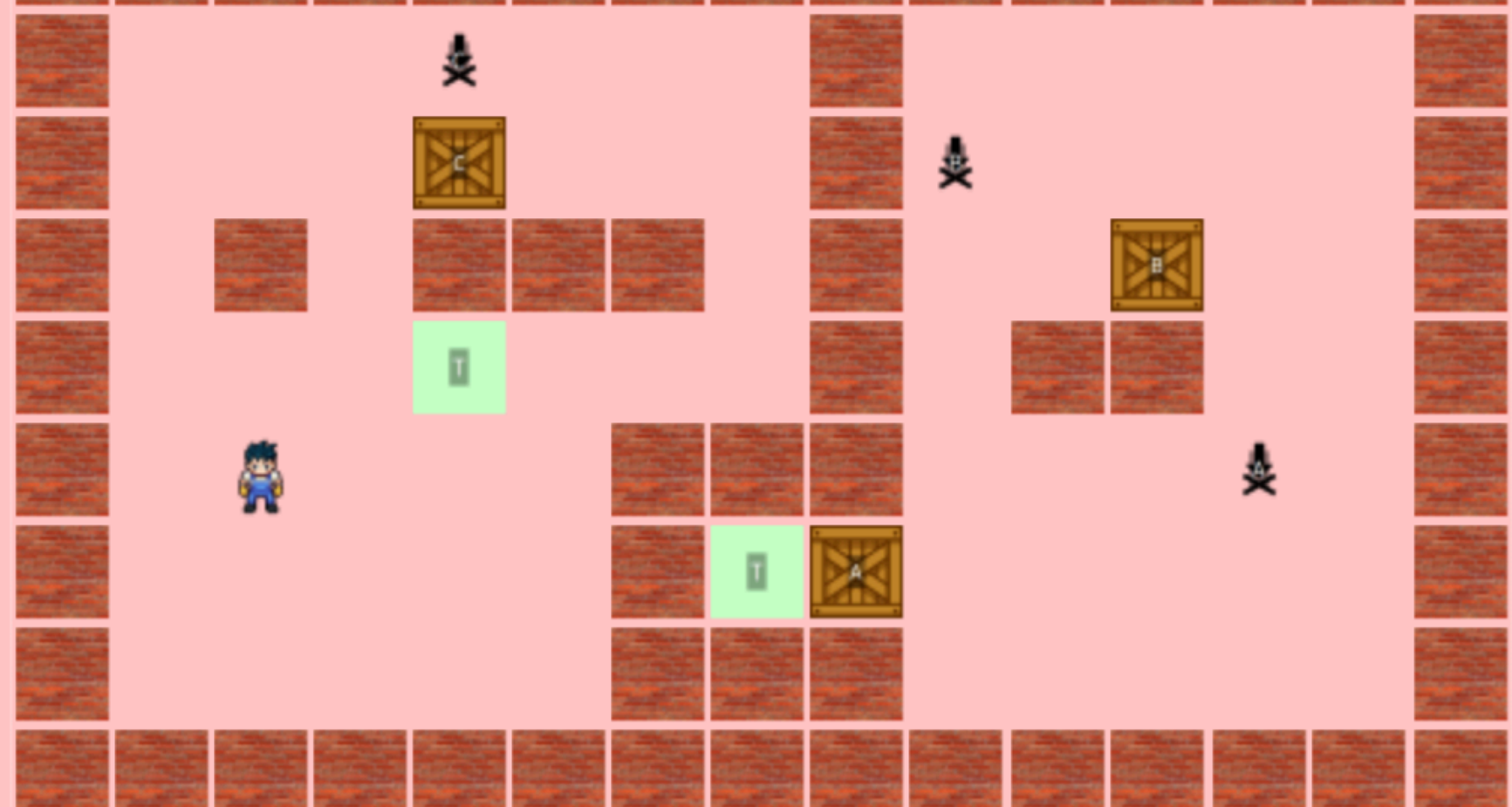
Internal DSL étendu
Nouvelles lettres détectées automatiquement :
a b c → boîtes
A B C → cibles

State Pattern
Une boîte peut être "bloquée / validée".

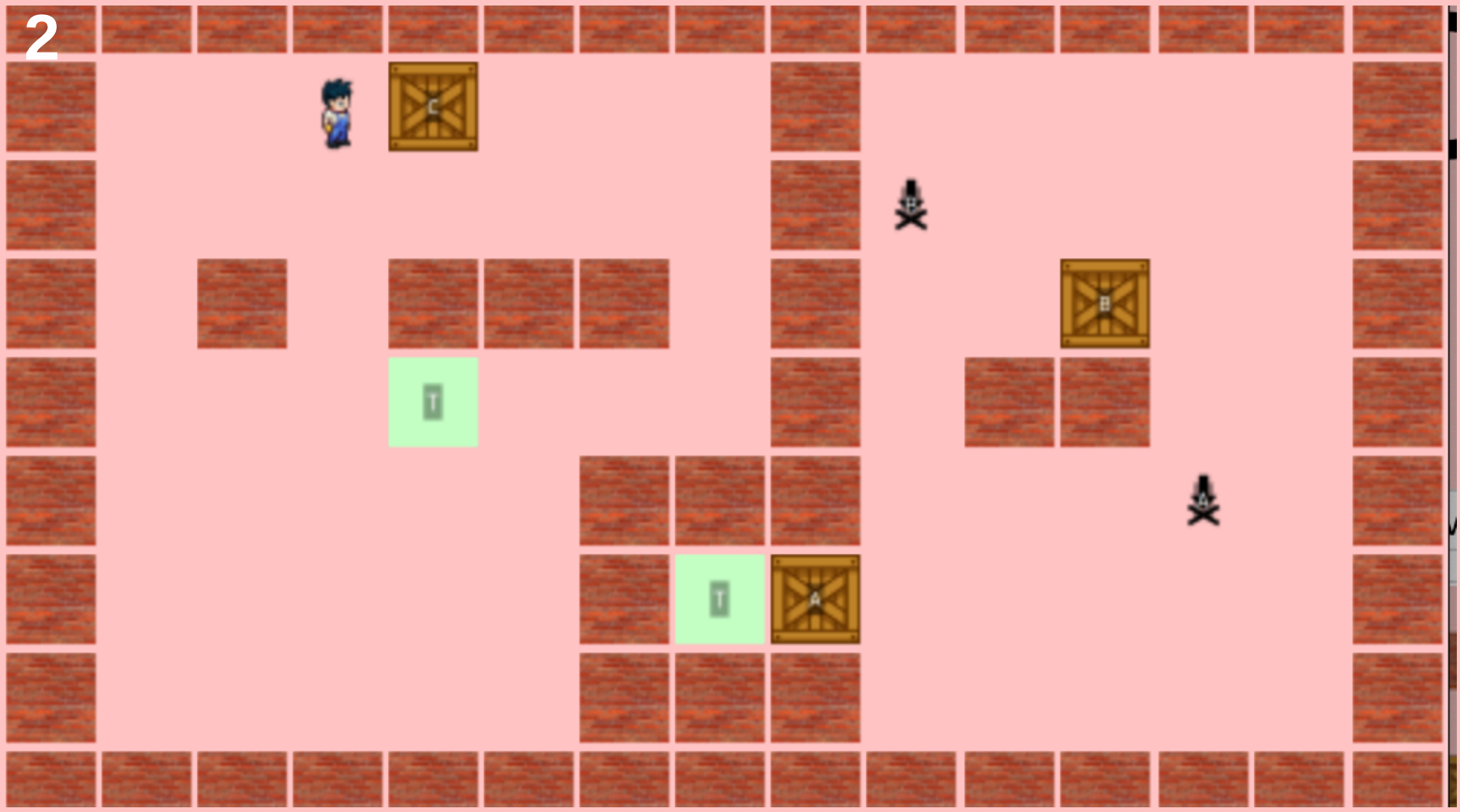
Tell-Don't-Ask
La boîte et la cible décident localement si la position est correcte.

```
#####  
#   A   B   #  
#  a   b   T  #  
#  ###  ###  #  
#  @   T     #  
#####
```

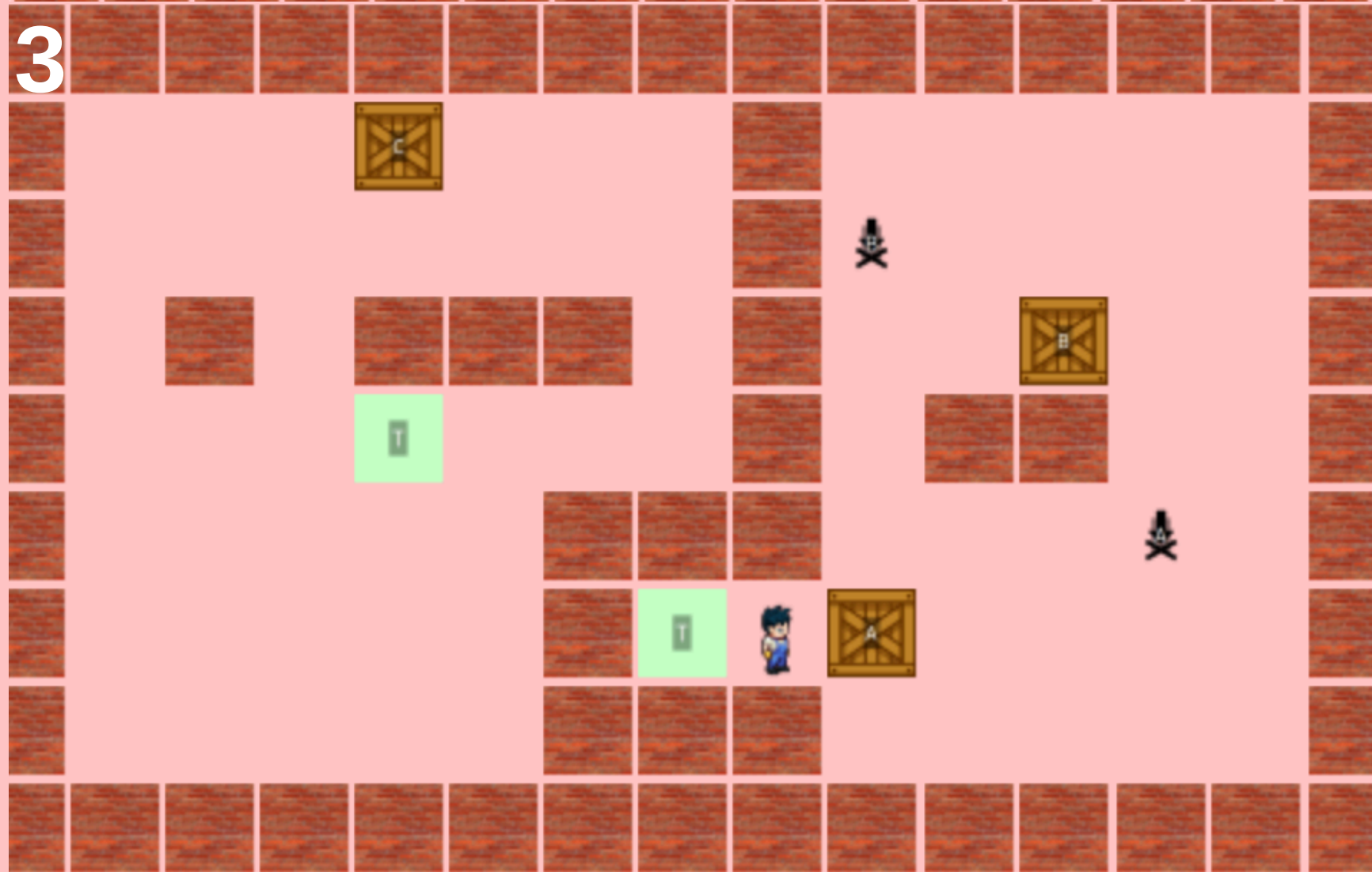

1



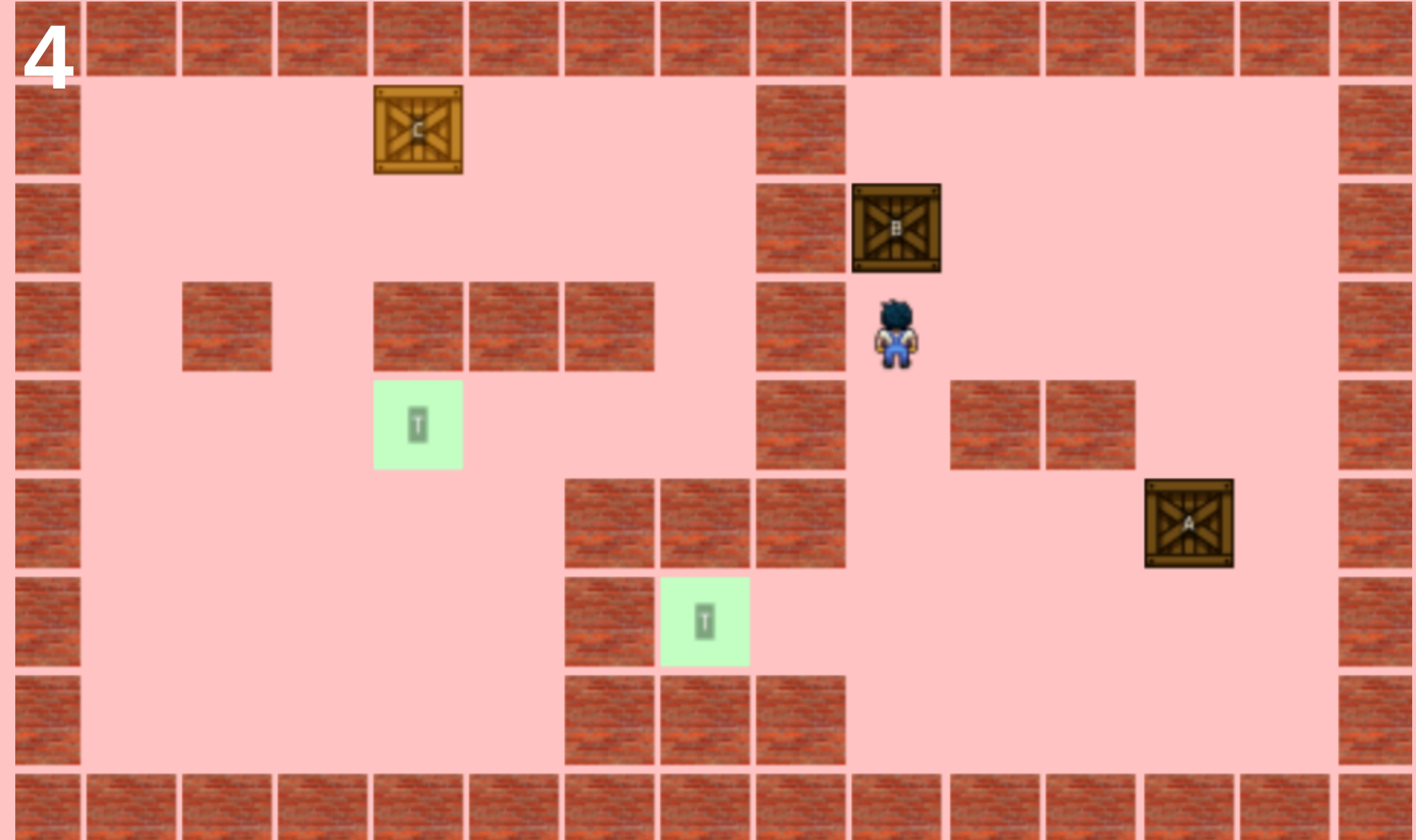
2

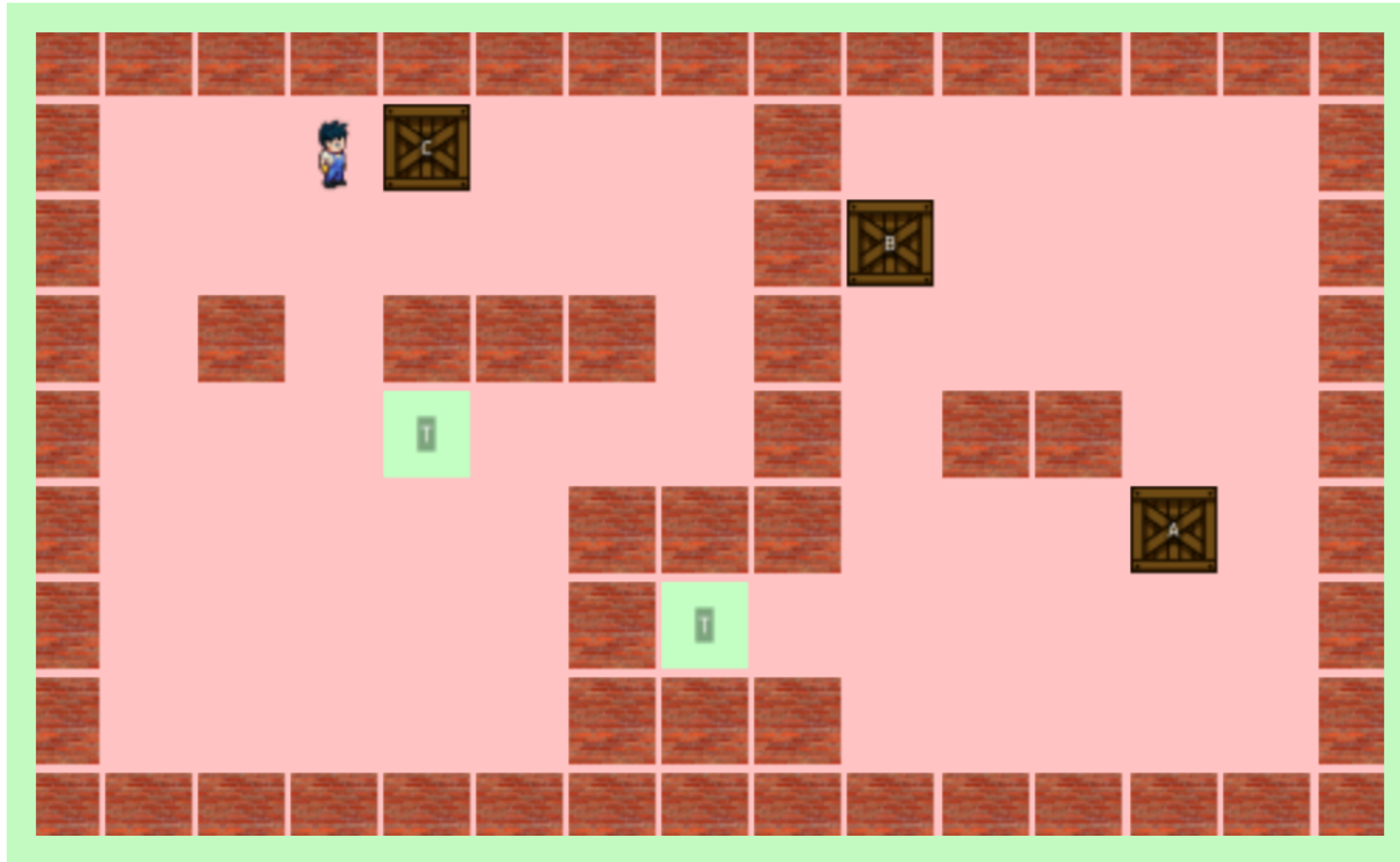


3

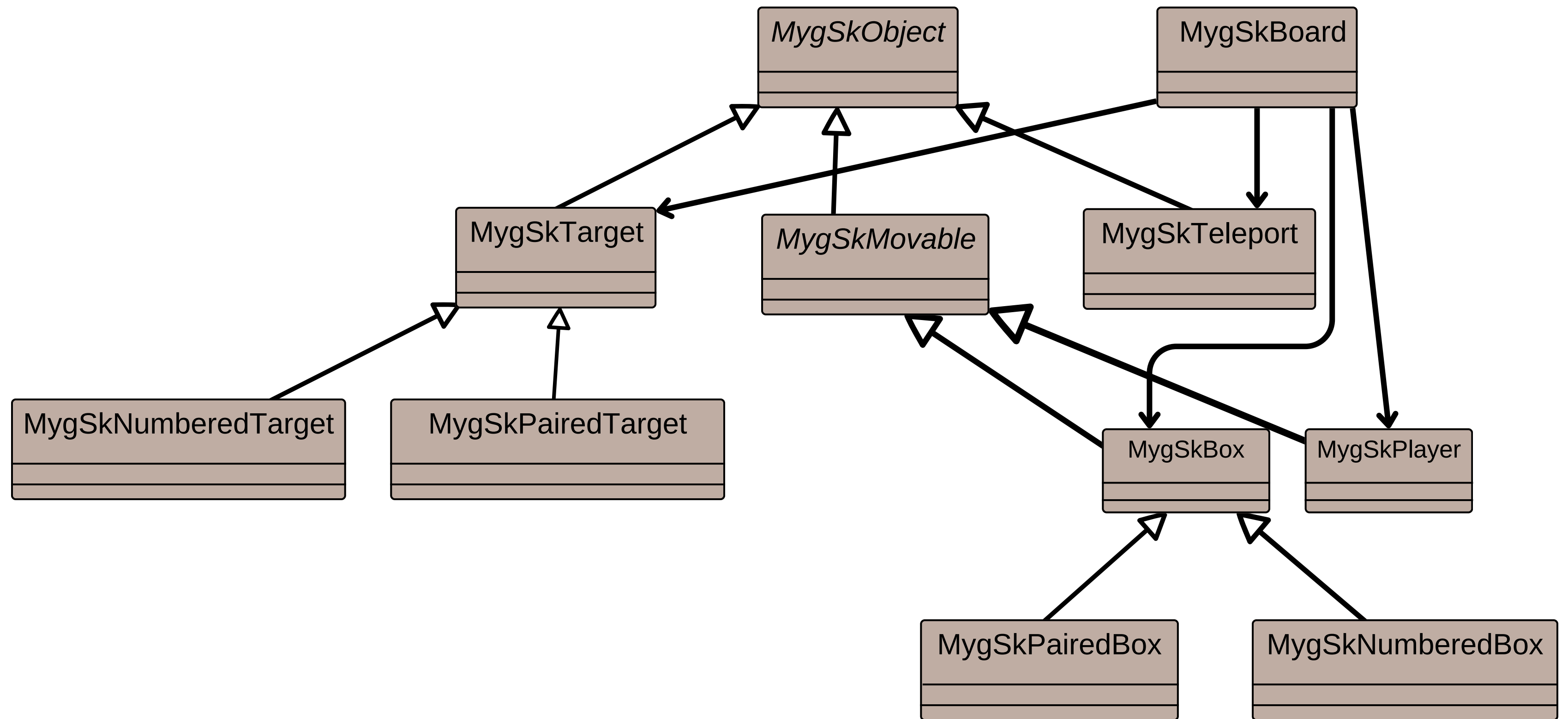


4





Nouvelles classes et modifications



Tests Automatiques

Objectif des tests : Assurer la stabilité et la fiabilité du moteur de jeu

Fonctionnalités testées

- ◆ **Téléporteurs**

Validation du comportement des paires, déplacement du joueur et non-téléportation des boîtes.

- ◆ **Compteurs** (déplacements & poussées)

Vérification que chaque action incrémente correctement les compteurs.

- ◆ **Boîtes numérotées** (A, B, ...)

Contrôle du verrouillage séquentiel et de l'état final du niveau.

- ◆ **Logique globale du plateau**

Tests d'état final, cohérence du fond, règles de Sokoban.

Améliorations possibles

1. Undo / Redo

- Historique des actions
- Annuler et refaire un coup
- Aide au débogage et au confort du joueur

2. Replay de la partie

- Sauvegarde des mouvements / poussées / téléportations
- Relecture automatique comme une démo
- Analyse et partage des solutions

3. Interface améliorée

- Téléporteurs visuellement liés
- Ordre des boîtes affiché (1-2-3 / A-B-C)
- Mini-carte
- Animations plus fluides



GAME
OVER

Merci pour votre attention!

